

Asymptotic

Interval geometry and
Mathics rewrite contracts

An incremental audit for Wolfram Language 15
and the repository’s Mathics3 compatibility layer

Repository	VladimirReshetnikov/Asymptotic
Reviewed revision	8cee870994f506b501bae3ea6bd4a3a7edb895c1
Commit timestamp	September 10, 2026, 17:10:56 UTC
Exclusion baseline	Five review waves; 35 retained reports and 10 retired-package tombstones
Retained audit items	Four distinct mechanisms, with different evidence and severity levels
Executed evidence	34 independent Python test methods; exact arithmetic, structural models, numerical-policy experiments, and patcher fixtures
Unexecuted	Wolfram/Mathics package runs, WL regression specifications, and integration of the candidate edits

Principal result. A zero-crossing odd interval power can lose its monotone endpoint geometry. In an explicit quartic inverse example, that loss changes a strictly positive derivative enclosure into an interval containing zero, even when the supplied center is the exact root. A bounded endpoint repair removes this obstruction in the independent rational model.

Additional results. The Mathics assumption protector rewrites bare membership symbols used as program data; numerical logarithm recovery uses rounded signs as branch premises; and a numerical-result description confuses the positive source coordinate with a signed power observable.

Contents

1	Executive assessment	2
2	Scope, sources, and nonduplication	3
2.1	What was inspected	3
2.2	Novelty boundaries	3
2.3	Evidence vocabulary	3
3	Architecture relevant to the new findings	4
4	N01: odd interval powers lose monotone endpoint geometry	4
4.1	The inspected algorithm	4
4.2	A certificate obstruction with an exact root	5
4.3	A narrow bounded-arithmetic repair	6
4.4	Tests, acceptance, and limitations	7
5	N02: assumption protection rewrites symbols used as data	7
5.1	The legitimate protection and the excessive match	7
5.2	A minimal program with no membership call	7
5.3	A surgical application-head repair	8
5.4	What this patch does not prove	8
6	N03: numerical log recovery needs exact branch premises	9
6.1	The recovery path and its premise	9
6.2	An exact negative family with misleading rounded signs	9
6.3	Why two negative factors are a branch error	10
6.4	A conservative exact positive grammar	10
6.5	Acceptance obligations	11
7	N04: local numerical metadata confuses root and observable	11
8	Executed experiments and what they establish	12
8.1	The saved test result	12
8.2	Interval oracle independence	12
8.3	Measured work is not native performance	13
8.4	Limits of the other models	13
9	Wolfram 15 and Mathics3 validation design	13
9.1	Separate platform obligations	13
9.2	Baseline, candidate, and control processes	14
9.3	A practical sequence	14
10	Targeted development opportunities	14
10.1	Distinguish geometric and arithmetic certificate limitations	14
10.2	Make assumption protection preserve a declared grammar	15
10.3	Require a proof premise for recovery identities	15
10.4	Make diagnostic examples include orientation	15
10.5	Implementation priority	15
11	Artifact guide and reproducibility	16
12	Conclusion	17
A	Source coverage record	17
B	Candidate edit boundaries	18

1 Executive assessment

The repository has a substantial remainder-aware symbolic architecture and an unusually extensive maintained review corpus. Its public interface, ordinary and specialized expansion engines, exact interval certification, numerical checks, generated standalone distribution, and Mathics adapters have already received multiple review waves. Repeating that backlog would obscure the incremental value of another audit. This report therefore concentrates on four mechanisms not found in the inspected exclusion indexes and the closest original reports. The reviewed revision is immutable; statements below do not describe an unspecified later `main`.[\[1, 2, 3, 4, 5, 6\]](#)

Item	Assessment	Consequence and evidence
N01	Medium: certificate coverage and unnecessary work	Odd interval powers can be safely but excessively wide. A precise derivative-separation counterexample is proved and reproduced in an independent rational model. The public package call remains unexecuted.
N02	Medium: Mathics assumption-program semantics	The protector replaces bare <code>Element</code> symbols inside assumption values, not just membership heads. The source transformation changes a concrete program's selected assumption. Its public expansion effect is a prediction.
N03	Medium: numerical recovery branch obligation	A machine-sign test is used to authorize an exact logarithm identity. An exact negative-factor family and a 53-bit arithmetic-policy model expose the missing premise. A current public fallback failure is not established.
N04	Low: diagnostic accuracy	The description of <code>LocalRoot</code> and <code>LocalApproximation</code> disagrees with the stored quantities for powered observables. The numerical formulas themselves are not challenged.

The most important distinction is between *a false enclosure* and *an enclosure too wide to finish a proof*. N01 is the latter. The existing interval algorithm still encloses the true power; its unnecessary width defeats a downstream derivative test. Similarly, N03 is not presented as a demonstrated false high-precision public answer. The exact branch premise is missing at the recovery helper, but constructor evaluation, the fallback trigger, and later realness checks can affect an actual public call.

The artifact bundle contains four narrow, source-anchored candidate edits, independent models and tests, raw WL characterization probes, desired regression specifications, and a novelty/coverage record. **All 34 independent Python test methods passed. No Wolfram or Mathics package execution succeeded in this environment.** A failed remote evaluator connection and absent local runtimes are limitations of this audit, not defects in the repository.

The immediate recommendation is to implement and characterize N01 and N02 first, then replace N03's numerical sign authorization with a small exact positive grammar, and correct N04's description without changing the established numerical fields. The later development recommendations in [Section 10](#) are restricted to these mechanisms rather than another general-purpose roadmap.

2 Scope, sources, and nonduplication

2.1 What was inspected

Source inspection used pinned GitHub connector reads. Direct cloning was unavailable because the container could not resolve the remote host. There was no complete local checkout. The reviewed files cover the public inverse constructor and object assembly, a substantial part of explicit series operations and composition, interval certificates, numerical checks, Fourier and native special-function paths, exact special-function identities, Dirichlet expansions, the standalone builder, and most of the small Mathics adapter modules. The coverage artifact records full reads separately from requested ranges whose tool output was truncated.[10, 11, 12, 18, 19, 20, 21, 16]

The exclusion screen used the corpus index, the main implementation register, the wave-3 and wave-4 intake crosswalks, the wave-5 index, and relevant original material from reviews 39, 42, and 45. In particular, the previous scalar interval-translation work, repeated affine recognition, powered-observable context, and recently narrowed Mathics numerical exception were considered before retaining the new items. This is *not* a claim that every page of all 35 retained articles was reread.[2, 3, 4, 5, 6, 7, 8, 9]

A GitHub symbol search returned no results even for an existing helper. It was therefore not used as evidence that a finding is absent. Negative novelty evidence comes from the maintained crosswalk and relevant original sources, not an unreliable empty code-search response.

2.2 Novelty boundaries

Item	Nearest prior records	Increment retained here
N01	C20, X05; R39 N02; R42 N02	The range of one scalar odd power across zero, with an explicit precision-independent derivative-separation obstruction. Neither an affine-recognition optimization nor a general request for stronger polynomial bounds.
N02	W3-02; W4-15	A replacement inside an already identified assumption value changes symbol-valued program data. Not the classification of option names or a generic rewrite-installation check.
N03	W3-10; W4-02; R45 N01	A <i>numerical recovery</i> helper uses a rounded sign to justify splitting a product logarithm. Not the monomial logarithm parser, root-finder precision guard, or exact-rational seed admission.
N04	C21; R43 F01	The prose describing two local numerical fields is wrong even when their computed values are correct. Not coefficient reconstruction or loss of a translated numerical displacement.

The IDs in this table are comparison anchors, not a reissued list of defects or recommendations. Broad issues already represented by the corpus were screened out rather than relabeled as discoveries. The resulting audit is intentionally narrower than a first review would be.

2.3 Evidence vocabulary

Source fact means a condition, assignment, or call path visible at the pin. *Deduction* means a mathematical or control-flow consequence with its premises stated. *Independent execution* means the supplied Python model ran. *Unexecuted characterization* means a WL program is supplied but no returned package result is claimed. *Candidate repair* means a proposed source change, not an accepted patch.

These distinctions are repeated at the places where they matter. A correct independent model is not a substitute for evaluator semantics, public dispatch, or integration testing. Conversely, a runtime outage does not prevent proving a mathematical width defect in an explicitly inspected interval formula.

3 Architecture relevant to the new findings

The ordinary inverse constructor retains an explicit forward function, source and target variables, the selected endpoint and side, observable power, coefficients, a remainder, and additional construction metadata. The `GeneralizedSeries` object exposes its finite expression through `Normal` and properties through association lookup. These retained facts are then consumed by operations and evidence routines; the latter do not reconstruct every premise from a displayed polynomial.[\[10, 18\]](#)

The certificate engine has a different contract from a numerical comparison. It uses exact rational interval endpoints with directed dyadic rounding, explicit bounds for exponential and logarithmic series, domain checks, a derivative enclosure separated from zero, and a residual argument. Its numerical seed merely proposes a center; exact interval arithmetic decides certification. This separation is why N01 can be analyzed without trusting a floating-point root solver.[\[11\]](#)

The ordinary numerical inverse checker instead solves the retained equation in a positive local coordinate. It checks the source domain and compares either the recovered source coordinate or a requested signed power observable. The new Mathics numerical module rewrites five package-owned numerical consumers and refuses precision that its underlying solver cannot substantiate, apart from its exact-seed exception. N03 concerns a later numerical-log recovery within that module, not the solver safeguard itself.[\[12, 14, 9\]](#)

Mathics support is implemented largely through private-context helpers selected during streaming package loading, with several late transformations of already parsed definitions. The official Wolfram path does not install the Mathics-only assumption protector or numerical-log recovery. This isolation makes N02 and N03 specifically Mathics-path issues, while N01 and N04 are present in shared source. A shared source path is not itself a cross-runtime acceptance result.[\[16, 10, 13, 14\]](#)

4 N01: odd interval powers lose monotone endpoint geometry

4.1 The inspected algorithm

`certIntegerPower` uses exponentiation by squaring. It multiplies the current answer interval by a base interval when the current exponent is odd, then squares the base interval. Its squaring step correctly recognizes a zero-crossing interval and uses lower bound zero instead of multiplying two independent copies. However, an odd power still combines the original zero-crossing interval with later nonnegative squared intervals as if their values were independent.[\[11\]](#)

The relevant structure is:

```
base = a; power = Abs[n]; answer = {1, 1};
If[n < 0, base = certReciprocal[base, ctx]];
While[power > 0,
  If[OddQ[power], answer = certMul[answer, base, ctx]];
  power = Quotient[power, 2];
  If[power > 0,
    base = certRoundInterval[
      If[base[[1]] <= 0 <= base[[2]],
        {0, Max[base[[1]]^2, base[[2]]^2}], Sort[base^2]], ctx]]];
```

answer

For $I = [-1/4, 1]$ and exponent three, the base square is $[0, 1]$. Multiplication then returns

$$I[0, 1] = [-1/4, 1].$$

But the real cube is increasing, so its exact range is

$$\{t^3 : t \in I\} = [-1/64, 1].$$

Both intervals are valid enclosures. Only the second retains the dependence between t and t^2 .

Proposition 4.1 (An arbitrarily large geometric overestimate). *Let $0 < a \leq b$ and let $n \geq 3$ be odd. Ignoring outward-rounding slack, the inspected binary interval algorithm applied to $[-a, b]^n$ returns*

$$[-ab^{n-1}, b^n],$$

whereas the exact hull is $[-a^n, b^n]$. The ratio of the magnitudes of the two lower endpoints is $(b/a)^{n-1}$.

Proof. The least significant bit is one, so the first answer interval is $[-a, b]$. Every subsequent squared base is $[0, b^{2^j}]$, since b has the larger magnitude. Multiplication by any selected such base multiplies both the negative and positive answer endpoints by its upper endpoint. The selected exponents sum to $n - 1$. Odd-power monotonicity gives the exact hull independently. $\square$

For $a = 2^{-m}$ and $b = 1$, all relevant endpoints are powers of two. The width discrepancy persists at every admitted significant-bit precision; it is not roundoff. In particular, increasing `EnclosureOrder` cannot fix this example's derivative-separation failure.

4.2 A certificate obstruction with an exact root

Consider

$$f(x) = \frac{(x-1)^4}{4} + \frac{x}{16}, \quad f'(x) = \frac{1}{16} + (x-1)^3. \quad (1)$$

Choose the source endpoint $x_0 = 2/3$, the increasing source side, verification interval $J = [3/4, 2]$, target $y = 1/16$, and center $c = 1$.

These choices avoid a branch-connectivity ambiguity. For all $x \geq 2/3$,

$$f'(x) \geq \frac{1}{16} - \frac{1}{27} = \frac{11}{432} > 0.$$

Thus f is strictly increasing on the whole selected source ray. Moreover,

$$f(2/3) = \frac{29}{648}, \quad f(1) = \frac{1}{16}, \quad f(1) - y = 0.$$

The center is an exact root and the entire verification interval lies on the selected side.

Over J , the affine expression $x - 1$ ranges over $[-1/4, 1]$. The old and sharp derivative enclosures are therefore

$$D_{\text{old}} = \frac{1}{16} + [-1/4, 1] = [-3/16, 17/16],$$

$$D_{\text{sharp}} = \frac{1}{16} + [-1/64, 1] = [3/64, 17/16].$$

The old interval contains zero; the sharp one has lower bound $3/64$.

In `certAttempt`, derivative separation is checked before the point residual is used. The inspected order of operations therefore predicts `DerivativeNotSeparated` for the old factored derivative expression, despite the exact center. With the sharp power enclosure, the residual radius is $0/(3/64) = 0$, and the root enclosure is the singleton $[1, 1]$.[\[11\]](#)

The public characterization supplied with the bundle is:

```
s = AsymptoticInverse[(x - 1)^4/4 + x/16,
  {x, 2/3}, {y, 3}, Direction -> "FromAbove"];
InverseCertificate[s, 1/16,
  "Interval" -> {3/4, 2}, "Center" -> 1,
  "EnclosureOrder" -> 2, "MaxRefinements" -> 0,
  "RefineExpansion" -> False]
```

Evidence boundary. This public sequence was not executed. The probe records `s["Function"]` because preservation of the factored source matters to the exact evaluation path. The ordinary constructor explicitly retains `"Function" -> f`; the direct power and derivative-enclosure witnesses isolate the mechanism independently of dispatch. The Python experiment reproduces the two derivative intervals and the exact residual, not a complete execution of `InverseCertificate`.[\[10, 11\]](#)

4.3 A narrow bounded-arithmetic repair

A tempting implementation computes the exact rational endpoint powers and rounds them once. That can introduce unnecessary large-integer work. A smaller change uses the existing rounded arithmetic separately on the two singleton endpoints:

```
If[n > 1 && OddQ[n] && a[[1]] < 0 < a[[2]],
  Return[{
    certIntegerPower[{a[[1]], a[[1]]}, n, ctx][[1]],
    certIntegerPower[{a[[2]], a[[2]]}, n, ctx][[2]]
  }, Module];
```

The insertion belongs after the existing absolute-exponent resource cap and before the reciprocal route. The singleton calls do not satisfy the strict zero-crossing condition, so the new branch does not recurse indefinitely. Negative powers retain the original zero-exclusion rule. Even powers, sign-definite intervals, endpoint-zero intervals, and exponent zero retain their existing paths.

Proposition 4.2 (Soundness of the candidate branch). *Assume the original singleton power routine encloses l^n and h^n with outward rounding, where $l < 0 < h$ and $n > 1$ is odd. Taking the lower endpoint of the first enclosure and the upper endpoint of the second encloses t^n for every $t \in [l, h]$.*

Proof. For odd positive n , $l^n \leq t^n \leq h^n$. Each singleton enclosure extends outward from its respective exact endpoint. Combining its lower and upper endpoints therefore preserves both inequalities. $\square$

The candidate has logarithmically many rounded multiplication/squaring steps in n , with approximately twice the existing arithmetic work on affected intervals. It does not claim constant bit complexity: rational magnitudes and dyadic exponents still matter. The original exponent cap remains in force. The benefit is geometric sharpness and avoidance of futile retries, not a claim that one power evaluation becomes faster.

4.4 Tests, acceptance, and limitations

Independent tests cover asymmetric and reflected cubes, an exact dyadic odd-power family, 500 seeded rational intervals with endpoint containment checks, 300 sign-definite unaffected cases, negative powers, singularity refusal, and the exponent cap. Directed rounding is checked on 1,000 rational inputs at three precisions. A large exponent exercises the logarithmic-step structure without timing the package.

For native acceptance, run the direct helper tests and the quartic certificate in fresh Wolfram 15 and Mathics processes, in both modular and generated layouts. Check unchanged even/sign-definite behavior, the zero-containing reciprocal failure, the original cap, and a case that genuinely fails derivative separation. Do not replace `DerivativeNotSeparated` with automatic success merely because this particular center is a root: uniqueness still depends on a valid derivative argument.

The proposed change does not solve general interval dependency, arbitrary polynomial range optimization, or other certificate failures. Those are outside this item's scope. The Mathics certificate adapter inspected at this pin changes a history assignment, not `certIntegerPower`; it does not already supply a different odd-power range algorithm.^[17]

5 N02: assumption protection rewrites symbols used as data

5.1 The legitimate protection and the excessive match

The Mathics assumption module protects membership predicates before ordinary option evaluation can weaken them. Its comment gives the example that a real-valued `Sin[a]` need not imply real a . Preserving the original membership question is useful. The late wrapper enters the package's held catch boundary and replaces membership symbols under syntactic `Assumptions` rule values.^[13, 15]

The overreach is in the candidate-position search:

```
heads = Position[held, System'Element,
  {0, Infinity}, Heads -> True];
```

It is followed by a prefix check establishing that the occurrence lies within an assumption rule's right-hand side. The name `heads` suggests application heads, but the pattern actually matches the symbol `System'Element` at any position, including an ordinary argument used as data. `Heads -> True` adds head positions; it does not restrict the search to them.^[13, 25]

This distinction matters because an option value can be a program that computes an assumption. Holding such a program prevents premature evaluation, but a structural rewrite can still change what it computes.

5.2 A minimal program with no membership call

Consider the held rule:

```
held = HoldComplete[Rule[Assumptions,
  If[Context[Unevaluated[System'Element]] === "System'",
    a > 0, a < 0]]];
```

The right-hand side asks for the context of a symbol; it does not ask whether any expression belongs to a set. Its intended result is $a > 0$. The old protector replaces that bare symbol with `AsymptoticAnalysis'Mathics'Element`, so the same context comparison selects $a < 0$. The string

literal `"System"` is not changed. The definition of `Context` and the two distinct symbol contexts establish the difference.[\[24, 13\]](#)

A direct characterization is:

```
protected = AsymptoticAnalysis`Private`mathicsProtectInputAssumptions[held];
{Last[ReleaseHold[held]], Last[ReleaseHold[protected]]}
```

The predicted pair is $\{a > 0, a < 0\}$ on the old Mathics path.

The possible mathematical consequence is straightforward. Under $a > 0$,

$$\sqrt{a^2} + x = a + x;$$

under $a < 0$, it is $-a + x$. An inline assumption program selecting the first condition must not be silently changed into one selecting the second. The supplied public probe applies the program to an expansion of `Sqrt[a^2] + x` with the package backend. **That public result is unexecuted**; it is not reported as an observed wrong coefficient.

The counterexample deliberately uses metaprogramming to expose the boundary precisely. Similar bare-symbol occurrences can appear in patterns such as `Blank[System`Element]`, symbolic dispatch tables, or quoted data. These examples are reasons to distinguish code and data, not evidence that ordinary declarative assumptions all fail.

5.3 A surgical application-head repair

Instead of matching all occurrences of the symbol, find actual two-argument membership applications and replace their head positions:

```
heads = (Append[#, 0] &) /@ Position[held,
  HoldPattern[System`Element[_, _]],
  {0, Infinity}, Heads -> False];
```

Keep the existing rule-RHS prefix filter and `ReplacePart` operation. The change fixes the bare-symbol witness, preserves protection of actual membership predicates, and leaves unrelated option values alone.

The independent structural model checks immediate and delayed assumption rules, real membership applications, bare symbols in the context program and in a pattern, unrelated rules, nested containers, wrong-arity applications, and idempotence. It models heads separately from arguments. It does *not* implement arbitrary WL evaluation, pattern matching, option resolution, or side effects.

5.4 What this patch does not prove

Replacing an actual membership application inside quoted code can still be observable. For example, a program may compare `HoldComplete[Element[a, Reals]]` with another held expression. The application-head patch still changes the quoted expression. The test suite explicitly preserves this limitation rather than declaring the repair fully hygienic.

Consequently, there are two distinct engineering decisions. The immediate patch should stop rewriting bare data. A later contract should decide which assumption-value grammar the protection promises to preserve: a declarative Boolean language, a narrowly interpreted delayed program, or arbitrary WL code. Arbitrary code cannot be made semantically transparent by a blanket symbol substitution. A refusal or a documented limitation is preferable to silently changing an unsupported program's meaning.

This is not a recommendation to remove the membership protector. Doing so would restore the very Mathics realness problem that motivated it. Acceptance must preserve the original declarative `Element[Sin[a], Reals]` protection while fixing the new witness. Native tests must also show that the official Wolfram path remains untouched.[\[13, 15, 10\]](#)

6 N03: numerical log recovery needs exact branch premises

6.1 The recovery path and its premise

The Mathics numerical module first evaluates `N[expr, precision]`. Only when its result contains `Indeterminate` does it try splitting product logarithms and reevaluating. The source comment describes a small-tail form such as `Log[Sqrt[Pi] 10^-20]`. The recovery helper uses:

```
mathicsNumericalSplitLog[factors_List] :=
  If[And @@ (TrueQ[N[#] > 0] & /@ factors),
    Total[Log /@ factors], Log[Times @@ factors]];
```

The sign checks have no requested precision argument; they use ordinary machine evaluation. More importantly, even a high-precision numerical sign would be evidence rather than an exact branch proof. The transformation changes an exact expression using a premise not proved about its exact factors.[\[14\]](#)

The inspected Mathics 10.0.1 numerical evaluator rounds number atoms and recursively processes numerical expressions. Its arithmetic source uses `mpmath.fsum` for machine sums and `mpmath.fprod` for machine products. These primary sources motivate the independent 53-bit arithmetic-policy experiment below. The experiment is not a full Mathics evaluation trace.[\[22, 23\]](#)

6.2 An exact negative family with misleading rounded signs

For $\varepsilon > 0$, define

$$q_\varepsilon = \sqrt{2} + \sqrt{3} - \sqrt{5 + 2\sqrt{6} + \varepsilon}. \quad (2)$$

Let $A = \sqrt{2} + \sqrt{3} > 0$. Since $A^2 = 5 + 2\sqrt{6}$,

$$q_\varepsilon = \frac{A^2 - (A^2 + \varepsilon)}{A + \sqrt{A^2 + \varepsilon}} = -\frac{\varepsilon}{A + \sqrt{A^2 + \varepsilon}} < 0. \quad (3)$$

This is an exact sign proof. It also supplies a stable high-precision evaluation that avoids subtracting nearly equal positive numbers.

The independently executed model rounds the radicals with 53-bit `mpmath` arithmetic and uses `fsum` to combine terms. With $\varepsilon = 10^{-30}$ and 10^{-40} it returns, in each case,

$$\tilde{q}_\varepsilon = 2^{-52} = 2.2204460492503130808 \dots \times 10^{-16} > 0.$$

The stable identity at 400-bit precision instead gives approximately

$$q_{10^{-30}} = -1.58918622597891 \dots \times 10^{-31}, \quad q_{10^{-40}} = -1.58918622597891 \dots \times 10^{-41}.$$

The saved evidence contains longer values and the exact precision settings. This is a concrete arithmetic-policy countermodel to treating a rounded sign as a mathematical premise, not a claim that every Mathics syntax spelling follows this sequence of rounding operations.

6.3 Why two negative factors are a branch error

Let $q = q_{10^{-30}} < 0$, $r = q_{10^{-40}} < 0$, and $s = 10^{-20} > 0$. Then $qrs > 0$, and its principal logarithm is real. But principal logarithms of the two negative factors satisfy

$$\begin{aligned}\text{Log } q &= \log |q| + i\pi, \\ \text{Log } r &= \log |r| + i\pi.\end{aligned}$$

Consequently,

$$\text{Log } q + \text{Log } r + \text{Log } s - \text{Log}(qrs) = 2\pi i. \quad (4)$$

No asymptotic limit or uncertainty is needed for this identity. The distinct factors avoid reducing the example syntactically to the square of one expression.

If an evaluator's machine sign test admits these exact negative factors as positive, the old helper is authorized by its own condition to perform the wrong branch transformation. The 400-bit experiment confirms the principal-log difference numerically, while Equation (4) proves it exactly.

Crucial limitation. This audit has not demonstrated that an actual package call on this family first returns **Indeterminate**, reaches the recovery, accepts the same rounded signs, and then escapes later realness checks. The full fallback trigger and public consequence are unexecuted. Some consumers may reject the introduced complex value rather than publish it. The retained item is therefore the recovery helper's unproved branch obligation, supported by an exact counterfamily and an upstream-informed numerical policy model, not a recorded false public result.

6.4 A conservative exact positive grammar

The recovery should split a product only when every factor has an exact positivity justification. A minimal candidate recognizes positive integers and rationals, named positive constants such as π and e , products of admitted positive factors, and rational powers of admitted positive bases. The candidate also includes the package's named Glaisher constant. No numerical sign is used.

For example:

```
mathicsNumericalPositiveFactorQ[e_] := Which[
  IntegerQ[e] || Head[e] === Rational, TrueQ[e > 0],
  e === Pi || e === E || e === System`Glaisher, True,
  Head[e] === Times,
  And @@ (mathicsNumericalPositiveFactorQ /@ List @@ e),
  Head[e] === Power &&
    (IntegerQ[e[[2]]] || Head[e[[2]]] === Rational),
  mathicsNumericalPositiveFactorQ[e[[1]]],
  True, False];
```

This grammar admits the intended factors $\sqrt{\pi}$ and 10^{-20} , as well as arbitrarily small positive exact rational factors without requiring their machine approximation to remain nonzero. It rejects the unknown-sign radical differences in Equation (2). An unrecognized factor leaves the logarithm unsplit; the existing failure remains honest.

The original successful first numerical evaluation is unchanged, because the recovery is not entered in that case. The tradeoff is narrower recovery for complicated but genuinely positive expressions. That loss is acceptable for an initial safety-oriented patch, but must be documented rather than called complete positivity support.

A general call to a simplifier is not automatically a solution: it must itself provide a trustworthy exact positivity conclusion on the relevant Mathics forms. A later extension can admit more algebraic factors using independently checked exact sign enclosures or algebraic root isolation. The design criterion is the branch premise, not the number of decimal digits requested from $\mathbb{N}$.

6.5 Acceptance obligations

Characterize the helper directly before attempting an end-to-end fallback. Record actual Mathics values of the two exact factors, the unsplit expression, the split expression, and the precision requested. Include positive small-tail controls, negative factors, zero, complex factors, and expressions whose sign remains unproved. Check that the five numerical consumers still preserve their original precision and realness gates.^[14]

Then establish one genuine fallback trigger separately. A passing direct helper test is not a claim that the package has successfully recovered a numerical result. Conversely, a refusal after the conservative patch is not evidence that the branch-safe change failed; compare whether the refused factor was inside the explicitly admitted positive grammar.

7 N04: local numerical metadata confuses root and observable

The ordinary checker writes the source chart as

$$x = x_0 + \sigma u, \quad u > 0, \quad \sigma \in \{-1, 1\}.$$

It solves for `localRoot`, a value of u . For `Power` $\rightarrow$ `r` with $r \neq 1$, the observable is the signed displacement power $(\sigma u)^r$. The source contains these assignments and output fields:^[12]

```
observed = If[power === 1, localRoot, (side localRoot)^power];
...
"LocalRoot" -> N[localRoot, wp],
"LocalApproximation" -> N[localApproximate, wp],
"LocalCoordinate" ->
"x = SourceOffset + SourceSide u; LocalRoot and
  LocalApproximation are values of u for Power 1
  and of u^Power otherwise."
```

The prose makes two errors. `LocalRoot` remains u for every power, not u^r . Also, `LocalApproximation` approximates $(\sigma u)^r$ when $r \neq 1$, which differs from u^r for a negative side and an odd integer power.

For an exact identity inverse on the negative side, choose target -2 and power three. The true local root is $u = 2$, the signed observable is $(-2)^3 = -8$, and the positive-coordinate cube is $2^3 = 8$. Thus the stored pair is conceptually

$$(\text{LocalRoot}, \text{LocalApproximation}) = (2, -8),$$

not two representations of the same positive-coordinate cube. Choosing an integer seed avoids conflating this diagnostic witness with the already reported exact-rational seed limitation.

The proposed edit changes only the string:

`LocalRoot` is always the positive source displacement u . `LocalApproximation` approximates u for power one and $(\text{SourceSide } u)^{\text{Power}}$ otherwise.

No stored number, source-side check, or observable convention should be altered. Keeping the fields stable avoids breaking consumers that already interpret the numbers correctly. A future machine-readable meaning tag would be useful only if it preserves this distinction; introducing another ambiguous field would be worse than correcting the prose.

This is a low-severity API/UX defect. The independent tests verify the signed-coordinate identities. The supplied native probe checks the actual result fields, but no returned package record is claimed.

8 Executed experiments and what they establish

8.1 The saved test result

The delivered run used Python 3.13.5 and mpmath 1.3.0. There are **34 passing test methods**, zero failures, zero errors, and zero skips. Of these, 28 test the independent mathematical/structural models and six test the diff emitter on synthetic source fixtures. No test in this count loads the repository in Wolfram or Mathics.

Group	Methods	Scope
Interval arithmetic	12	Directed dyadic rounding, containment, odd-power geometry, unaffected paths, singularity/cap controls, and the quartic derivative stage.
Assumption rewrite	8	A bounded AST model of occurrence replacement, application-head replacement, scope, idempotence, and the remaining quotation limitation.
Log recovery	6	Exact-sign counterfamily, 53-bit policy model, principal-log branch difference, and conservative positive-grammar controls.
Numerical metadata	2	Positive-coordinate roots versus signed powered observables.
Patch emitter	6	Unique/missing/ambiguous anchors, read-only behavior, all-or-nothing diff output, and default revision-check behavior.

Nested deterministic loops are described separately in `independent-results.json`. They are not inflated into an additional count of package test cases. The output file also records that native execution, WL candidate execution, and complete upstream source matching did not occur.

8.2 Interval oracle independence

The tested power implementation follows the inspected binary interval control structure. Its correctness oracle does not: the oracle computes the mathematical endpoint hull using odd-power monotonicity and the even-power minimum at zero. This is a useful independence boundary. Merely comparing two transcriptions of the same recurrence would not establish containment or expose its geometric loss.

The outward-rounding function is checked separately, including negative rationals. Integer-length selection, floor/ceiling direction, and zero handling are exercised before conclusions are drawn about powers. The source's significant-bit budget is retained; the model does not relabel decimal arithmetic as exact intervals.

8.3 Measured work is not native performance

The model records multiplication and squaring counts for several exponents. In the affected interval $[-1/4, 1]$, the endpoint repair calls the old singleton algorithm twice. For example, exponent three changes two multiplication calls and one square into four multiplication calls and two squares. At exponent 99,999 the number of steps is still logarithmic in the exponent.

These counts intentionally show the local cost increase. They do not establish a package speedup. The practical opportunity is that a sharper derivative bound can eliminate unsuccessful certificate attempts or unnecessary subdivision. Measuring that end-to-end benefit requires actual kernel runs and a corpus of certificate requests, with arithmetic and geometric failures reported separately.

8.4 Limits of the other models

The AST model has symbol tokens, application heads, arguments, and a tiny evaluator for the specific context-selection witness. It is not a WL interpreter. The floating experiment reproduces one precision policy using the upstream arithmetic primitives, not the complete Mathics conversion and simplification pipeline. The metadata model is an algebraic identity check, not a numerical solver.

The candidate diff emitter was tested against miniature fixtures containing the inspected source anchors. It was not run against a complete checkout in this environment. It requires the reviewed Git revision by default and requires each anchor to occur exactly once. Those guards reduce accidental application to an unrelated source tree; they do not certify that a generated edit is mathematically or evaluator-correct.

9 Wolfram 15 and Mathics3 validation design

9.1 Separate platform obligations

Item	Wolfram 15	Mathics3
N01	Execute direct power/derivative probes and public certificate; preserve existing interval controls.	Repeat shared-path probes and verify that the compatibility context does not change return or failure behavior.
N02	Verify that the Mathics-only protector remains absent from the active public route.	Establish actual held-program behavior, membership protection, once-only option evaluation, and the quotation boundary.
N03	Verify unchanged native numerical dispatch.	Characterize actual primitive signs, helper output, a genuine fallback trigger, and conservative positive controls.
N04	Check field values and corrected description on positive/negative sides and powers one, two, and three.	Repeat admitted exact-integer controls without treating unsupported high precision as a pass.

The repository's current compatibility source and historical records specifically discuss Mathics 10.0.1. This report uses that version as the upstream source reference; it does not claim that 10.0.1 is the newest release or that every Mathics version has identical behavior. Likewise, no local Wolfram 15 version was observed because the remote evaluator failed before returning a version string.[\[1, 14, 22, 23\]](#)

9.2 Baseline, candidate, and control processes

Use a fresh process for the pinned baseline and a separate fresh process for candidate sources. Late definition rewrites make a single-session overlay comparison difficult to interpret. After applying reviewed edits to canonical modules, rebuild the standalone file with the repository’s builder, then run the same focused probes for both distribution layouts. The builder exists to preserve streaming context behavior; editing only the generated artifact is not a canonical source update.[21, 10]

The supplied `native_probe.wl` prints the actual kernel version, package path, raw observations, and explicit start/end markers. It does not verify the checkout revision by itself and does not infer a pass from reaching the end. The expected pin printed by the file is plainly labeled as expected, not observed. Record the actual checkout revision externally.

The `desired_regressions.wlt` file supplies common desired contracts and guarded Mathics-only cases. It is an unexecuted specification. Its existence is not a claim that Mathics implements the full Wolfram MUnit runner. Use the appropriate supported harness rather than counting inert `VerificationTest` expressions as executed tests.

9.3 A practical sequence

First establish the N01 direct cube and derivative interval, since neither requires inverse construction. Next inspect the stored forward expression and run the public exact-center certificate. Then characterize N02’s held tree before evaluating its selected assumption. For N03, separate the sign primitive, direct split helper, and actual `Indeterminate`-triggered recovery. Finally verify N04’s numbers and text without changing the observable convention.

This ordering localizes failures. For example, a constructor refusal should not be described as a refutation of the independent interval-width computation. A correct direct split helper should not be described as evidence that a numerical fallback succeeded. A metadata assertion should not be allowed to hide an earlier numerical-precision refusal.

10 Targeted development opportunities

10.1 Distinguish geometric and arithmetic certificate limitations

N01 provides a concrete reason to distinguish two kinds of interval uncertainty. Directed rounding and truncated transcendental series create arithmetic uncertainty that more precision can reduce. Reusing an interval as independent copies creates geometric dependency loss that more precision need not reduce at all.

A useful incremental certificate diagnostic would record which elementary operation first blocked derivative separation and whether its enclosure changes under a precision increase. For the quartic witness, the exact old cube remains $[-1/4, 1]$ at every precision. Reporting this stable geometric obstruction would guide a user toward a sharper range evaluation rather than another expensive precision request.

This proposal is intentionally local to elementary-power range provenance. It is not a replacement for the already recorded general certificate roadmap. A small first implementation can tag the odd-power endpoint branch and retain its input/output intervals, with no new global theorem-prover machinery.

10.2 Make assumption protection preserve a declared grammar

N02 suggests a precise contract boundary: which expression forms are conditions to be protected, and which are programs or data that must retain their identity? The immediate application-head patch can be accepted independently of a larger language design. Its tests should become examples in the assumption documentation, including one bare-symbol program and one real membership predicate.

For further development, prefer a small declarative grammar with explicit interpretation over increasingly broad symbol replacement. A delayed assumption is not necessarily declarative merely because it eventually returns a Boolean expression. Where arbitrary code is intentionally supported, the implementation should preserve its execution and then protect only the resulting condition at a controlled boundary, without pretending it can reconstruct membership semantics already changed by caller-side evaluation.

That last point is a real tradeoff. Waiting for arbitrary code to evaluate can expose Mathics's native membership simplification; rewriting arbitrary held code can change the program. The correct solution must state its supported language rather than silently choosing one risk.

10.3 Require a proof premise for recovery identities

N03 is a case of a numerical recovery routine changing exact symbolic structure. Such recovery should have a small explicit contract: the identity used, its branch premises, and the conservative behavior when those premises are unproved. The initial positive-factor grammar supplies this contract cheaply for the intended small-tail example.

An extension to exact algebraic factors should return an independently checkable sign conclusion or remain unresolved. The important development opportunity is not simply more digits or a more aggressive simplifier, but admission based on exact sign information. Keep the first successful numerical route unchanged and benchmark recovery only on expressions that actually enter it.

10.4 Make diagnostic examples include orientation

N04 shows that scalar numerical fields need examples with negative source sides and odd powers. A documentation example with only $\sigma = 1$ and $r = 1$ cannot distinguish source roots, local displacements, and observables. Add a small table using $(\sigma, r, u) = (1, 2, 2)$ and $(-1, 3, 2)$ to the numerical-check documentation. Preserve the existing legacy fields; explain exactly what each contains.

10.5 Implementation priority

Order	Change	Completion criterion
1	N01 endpoint geometry	Direct helper and quartic certificate pass on both runtimes and both layouts; unchanged-path controls remain valid.
2	N02 application-head restriction	Bare-symbol program is unchanged; real membership remains protected; unsupported quotation behavior is explicit.
3	N03 exact-positive recovery	Intended positive factors recover where supported; no approximate sign authorizes an exact split; public fallback evidence is recorded separately.
4	N04 diagnostic correction	Values remain unchanged and the description matches positive-coordinate and signed-observable controls.

Priorities here are relative to these four audit items, not an attempt to reorder the entire existing

project backlog. Any runtime characterization that reveals a broader concrete failure should be recorded separately with its actual input and result.

11 Artifact guide and reproducibility

Path	Purpose
article.tex, article.pdf	Complete article source and compiled document.
code/independent_models.py	Directed rational interval model, bounded AST model, exact-positive grammar, and numerical-policy experiment.
code/test_independent.py	28 independent test methods.
code/test_patch_emitter.py	Six synthetic fixture tests for the read-only diff emitter.
code/run_independent.py	Runs all tests and writes the independent evidence receipt.
patches/emit_candidate_patch.py	Emits source-anchored candidate diffs; never edits the checkout.
code/native_probe.wl	Unexecuted raw characterization probes.
code/desired_regressions.wlt	Unexecuted desired helper and public contracts.
evidence/	Test output, witnesses, environment, novelty ledger, source coverage, and document preflight.

Run the independent checks with Python 3.10 or later and mpmath 1.3.0:

```
python code/run_independent.py
```

This overwrites its own independent receipt and test-output files. It does not use the network, load the Wolfram package, or change an upstream checkout.

To inspect candidate edits against an existing checkout:

```
python patches/emit_candidate_patch.py /path/to/Asymptotic
python patches/emit_candidate_patch.py /path/to/Asymptotic --which power
```

The default checks Git HEAD against the reviewed revision. The explicit `-skip-commit-check` option permits a deliberately selected unversioned fixture or changed tree; source anchors must still match exactly once. A failure emits no partial diff. Applying any emitted diff remains a separate user action and requires subsequent native validation.

To run raw package characterizations in a fresh kernel:

```
$AuditPackagePath = "/absolute/path/to/Asymptotic/src/Kernel/AsymptoticAnalysis.wl";
Get["/absolute/path/to/bundle/code/native_probe.wl"];
```

The probe prints observations rather than manufacturing a JSON success receipt. Preserve baseline and candidate output separately. Its expected revision line is not a source fingerprint or proof of what was loaded.

The article is self-contained LaTeX and can be rebuilt by running `pdflatex` three times or using `build.sh`. The archive contains no checksum files and no standalone font files. Embedded PDF fonts are sufficient for reading.

12 Conclusion

The useful incremental result is a small set of well-separated mechanisms, not another restatement of the broad backlog. N01 proves that a safe scalar interval primitive can still obstruct a certificate for an exact root through geometric dependency loss. N02 shows that protecting a mathematical predicate must not rewrite a symbol merely because it occurs as data inside an assumption program. N03 isolates an exact branch obligation hidden in numerical error recovery. N04 corrects a diagnostic description while preserving the established observable convention.

The supplied repairs are narrow and the independent evidence is reproducible. Their remaining boundary is equally explicit: they have not been integrated and executed in Wolfram or Mathics. The next acceptance step is therefore the focused, fresh-process runtime matrix in Section 9, not treating the independent test count as package validation.

A Source coverage record

The machine-readable coverage file is the authoritative detailed inventory. The following summary identifies the areas inspected and their limitations.

Area	Source material	Coverage boundary
Public inverse and object assembly	<code>AsymptoticAnalysis.wl</code> : constructor, result assembly, object access, late loader	Selected large ranges; not the full core or standalone
Exact certification	<code>InverseCertificates.wl</code> ; <code>MathicsCertificate.wl</code>	Main arithmetic, attempt, and refinement paths; some response truncation
Numerical evidence	<code>NumericalInverseChecks.wl</code> ; <code>MathicsNumerical.wl</code>	Complete modules
Assumption and compatibility helpers	Mathics assumptions, input protection, simplification, compatibility, calculus, algebra, Taylor, time budget, core functions	Complete small modules
Series operations	<code>SeriesOperations.wl</code> ; <code>SeriesEnvelopeArithmetic.wl</code>	Large selected ranges; not every operation
Special functions	Dirichlet, parameterized, exact identity, native-import and inverse-adapter modules	Complete small modules; selected native/adapter ranges
Fourier machinery	<code>FourierCoefficients.wl</code>	Initial coefficient and constructor ranges
Distribution	<code>validation/build_standalone.py</code> ; source-module README	Complete builder; no full local assembly run
Exclusion corpus	Main index/register, wave-3/4/5 material, closest originals	Crosswalk-led screen, not all articles in full
Mathics implementation reference	10.0.1 <code>nevaluator.py</code> , <code>arithmetic.py</code>	Selected upstream source ranges; no installed interpreter

The review does not claim absence of other defects, exhaustive feature coverage, or current full compatibility. Existing records already discuss those broad obligations. This appendix describes the evidence gathered here rather than converting earlier receipts into a new acceptance claim.

B Candidate edit boundaries

The power edit inserts one special case into the canonical certificate primitive. It does not replace the certificate engine. The assumption edit changes candidate replacement positions, leaving the existing RHS scope filter. The log edit adds a conservative exact-positive predicate and replaces only its recovery admission test. The metadata edit changes one string and no numerical computation.

All four are emitted as textual diffs only after unique-anchor checks. No complete upstream source file is bundled as a substitute for the repository. The code and article use the same revision pin. Native public probes and desired tests are deliberately kept separate from the executed evidence files.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, pinned repository README, revision 8cee870994f506b501bae3ea6bd4a3a7edb895c1. [README.md](#).
- [2] *Retained code-review corpus index*. [external-reports/code-review/README.md](#).
- [3] *Code review status and implementation register*. [CODE_REVIEW_STATUS.md](#).
- [4] *Wave 3 intake and finding crosswalk*. [WAVE_3_INTAKE.md](#).
- [5] *Wave 4 intake and finding crosswalk*. [WAVE_4_INTAKE.md](#).
- [6] *Wave 5 retained review index*. [wave-5/README.md](#).
- [7] *Review 39: incremental repository audit*. [Review 39 README](#).
- [8] *Review 42: regularity contracts and structural work*. [Review 42 README](#).
- [9] *Review 45: exact-root admission*. [Review 45 README](#); article source.
- [10] *Public kernel and inverse construction*. [src/Kernel/AsymptoticAnalysis.wl](#).
- [11] *Exact rational inverse certificates*. [src/Kernel/InverseCertificates.wl](#).
- [12] *Numerical inverse checks*. [src/Kernel/NumericalInverseChecks.wl](#).
- [13] *Mathics held input-assumption protection*. [src/Kernel/MathicsInputAssumptions.wl](#).
- [14] *Mathics numerical admission and logarithm recovery*. [src/Kernel/MathicsNumerical.wl](#).
- [15] *Mathics finite-real and sign assumption helpers*. [src/Kernel/MathicsAssumptions.wl](#).
- [16] *Mathics private compatibility helpers*. [src/Kernel/MathicsCompatibility.wl](#).
- [17] *Mathics certificate assignment adapter*. [src/Kernel/MathicsCertificate.wl](#).
- [18] *Explicit series operations*. [src/Kernel/SeriesOperations.wl](#).
- [19] *Exact special-function identities*. [src/Kernel/SpecialFunctionIdentities.wl](#).
- [20] *Dirichlet special-function expansions*. [src/Kernel/DirichletSpecialFunctions.wl](#).

- [21] *Standalone distribution builder*. [validation/build_standalone.py](#).
- [22] Mathics3 contributors, *Numerical evaluation*, tag 10.0.1.
<https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/eval/nevaluator.py>.
- [23] Mathics3 contributors, *Arithmetic evaluation*, tag 10.0.1.
<https://github.com/Mathics3/mathics-core/blob/10.0.1/mathics/eval/arithmetic.py>.
- [24] Wolfram Research, *Context*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Context.html>.
- [25] Wolfram Research, *Position*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Position.html>.